S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

Practical No. 10

Aim: Apply prompt engineering to construct a prompt for learning the concept of Adversarial Search.

Name of Student:

Roll No.:

Semester/Year: V/III

Academic Session:

Date of Performance:

Date of Submission:

AIM: To design and refine an effective prompt using prompt engineering techniques for learning the concept of Adversarial Search from Artificial Intelligence.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To be able to understand the concept of a *prompt* (input given to an AI model).
- To be able to understand the concept of *prompt engineering* (designing inputs to get desired outputs)
- To be able to work with basic idea and models like ChatGPT.
- To be able to distinguish between a vague prompt and a well-structured one.

THEORY:

i. Fundamentals of Prompt Engineering

Prompt engineering is the process of designing and structuring inputs (prompts) to get accurate, relevant, and well-formatted outputs from AI systems such as ChatGPT. Prompt engineering is how you ask a question to AI so that it gives the best possible answer.

ii. Characteristics of a Good Prompt

A good prompt is based on the following core principles:

- **Clarity** → Avoid ambiguity
- **Specificity** → Define exact task
- **Context** → Provide background
- **Constraints** → Limit output (length, format, style)
- **Intent** → Clearly state what is required

Example:

- Weak: *"Explain AI"*
- Strong: *"Explain AI in 5 bullet points with real-world examples for beginners"*

iii. Prompt Patterns / Techniques

The common patterns of the prompt along with their usage are as follows:

a) Role-Based Prompting

"You are an AI professor…"

b) Step-by-Step Prompting

"Explain step-by-step…"

c) Few-shot Prompting

Provide examples in the prompt

d) Instruction + Constraints

"Explain in 100 words using simple language"

e) Chain-of-Thought (Guided Reasoning)

"Show reasoning before final answer"

iv. Task-Oriented Prompting

Based on the tasks we can have different ways to provide the prompt. Following are the examples to design prompts for different tasks:

- **Learning/Explanation**
- **Coding**
- **Summarization**

- **Question generation**
- **Problem solving (AI, Cloud, etc.)**

v. Iterative Refinement (Very Important)

The first prompt may not be the best one. So, we must learn how the prompt can be refined to get most appropriate outcome:

- First prompt ≠ best prompt
- Improve by:
  - Adding constraints
  - Asking follow-up questions
  - Correcting output

Concept:

Prompt → Output → Refine → Better Output

vi. Evaluation of AI Output

It is very important to evaluate the output generated by AI. Critical thinking should be done to find the answer to the following:

- Is the answer **correct**?
- Is it **complete**?
- Is it **biased or vague**?
- Does it follow instructions?

vii. Ethical and Responsible Use

- Avoid Misuse and Plagiarism
  - Do not copy AI-generated content blindly for assignments or research
  - Always ensure originality and proper attribution when required
- Verify Information
  - AI tools like ChatGPT may generate incorrect or outdated information
  - Cross-check important facts before using them in academic or professional work
- Use AI as a Support Tool
  - AI should assist learning, not replace critical thinking
  - Students should understand concepts rather than depend entirely on AI-generated answers

viii. Basic Applications in Academics

- Generate notes
- Create MCQs and scenario-based questions
- Debug code
- Explain complex topics simply

ix. Perform the following tasks:

- Convert a bad prompt → good prompt
- Generate:
  - Notes
  - Quiz questions
  - Code snippets
- Design a prompt for:
  - Adversarial Search explanation

EXPECTED OUTPUT:

- Definition
- Game tree explanation
- Minimax working
- Alpha-Beta optimization
- Example
- Complexity analysis

INPUT & OUTPUT:
Design the prompts and provide the analysis using the following criteria:

Attempt	Prompt Quality	Output Quality	Improvements
1	Basic	Vague	Add structure
2	Improved	Better	Add constraints
3	Refined	High quality	Final

"Explain adversarial search in Artificial Intelligence using Rock–Paper–Scissors as an example. Describe minimax algorithm and alpha-beta pruning."

Share

1. Definition of Adversarial Search

Adversarial search is a technique used in **Artificial Intelligence (AI)** for decision-making in situations where **two or more players compete against each other.** In such problems, one player tries to **maximize their advantage**, while the opponent tries to **minimize it.**

These problems are commonly seen in **games such as chess, tic-tac-toe, and rock–paper–scissors.** AI must consider the possible actions of the opponent before deciding the best move. Therefore, the search process analyzes **all possible future game states** to determine the optimal strategy.

---

2. Game Tree Explanation

A **game tree** is a graphical representation of all possible moves in a game.

- The **root node** represents the current state of the game.
- Each **branch** represents a possible move by a player.
- The **child nodes** represent the resulting game states after each move.
- **Leaf nodes** represent the final outcomes such as win, loss, or draw.

For **Rock–Paper–Scissors**, each player has three choices:

- Rock
- Paper
- Scissors

If the AI chooses **Rock**, the opponent can choose **Rock, Paper, or Scissors.** Each combination leads to a different result (win, lose, draw). These possibilities fo a **game tree** that AI analyzes before selecting the best move.

Ask anything

ChatGPT

Share

3. Minimax Algorithm

The **Minimax algorithm** is used in adversarial search to determine the **best possible move** for the AI assuming that the opponent also plays optimally.

The idea behind minimax is:

- **Max player (AI)** tries to **maximize the score**.
- **Min player (opponent)** tries to **minimize the score**.

Steps of the Minimax Algorithm:

1. Generate the **game tree** for all possible moves.
2. Assign **utility values** to the leaf nodes (for example: win = +1, draw = 0, loss = −1).
3. Propagate the values upward in the tree:
   - At **Max nodes**, choose the maximum value.
   - At **Min nodes**, choose the minimum value.
4. The move with the **best value at the root node** is selected as the optimal move.

Example in Rock–Paper–Scissors:

- If AI chooses **Rock**:
  - Opponent chooses **Paper → AI loses (-1)**
  - Opponent chooses **Scissors → AI wins (+1)**
  - Opponent chooses **Rock → Draw (0)**

Minimax assumes the opponent chooses the **worst possible outcome for AI**, so AI evaluates moves carefully to choose the safest option.

Share

4. Alpha–Beta Pruning

Alpha–Beta pruning is an optimization technique used to **improve the efficiency of the minimax algorithm**.

It works by **eliminating branches of the game tree that do not affect the final decision.**

Two parameters are used:

- **Alpha (α):** The best value that the **maximizing player** can guarantee so far.
- **Beta (β):** The best value that the **minimizing player** can guarantee so far.

If at any point $\alpha \geq \beta$, the remaining branches of that node are **pruned (ignored)** because they cannot influence the final decision.

Benefits:

- Reduces the number of nodes evaluated
- Speeds up the decision-making process
- Allows deeper search in complex games

---

5. Example of Minimax with Alpha–Beta Pruning

Consider the AI deciding its move in **Rock–Paper–Scissors.**

Possible outcomes:

AI Move	Opponent Move	Result	Utility
Rock	Scissors	Win	+1
Rock	Paper	Loss	-1
Rock	Rock	Draw	0

During evaluation:

- If the algorithm already finds a move giving **+1**, and another branch cannot produce a better result, **alpha–beta pruning stops exploring that branch.**
- This reduces unnecessary calculations and speeds up the decision.

"Explain adversarial search in Artificial Intelligence using Rock–Paper–Scissors as the example game. Include definition of adversarial search, explanation of the game tree, working of the minimax algorithm, alpha-beta pruning optimization, a simple example, and complexity analysis."

1. Definition of Adversarial Search

Adversarial search is a technique used in **Artificial Intelligence (AI)** to solve problems where **two or more players compete against each other.** In such situations, each player tries to **maximize their own advantage** while minimizing the opponent's advantage.

This type of search is commonly used in **game-playing AI**, where the system must consider the **possible actions of an opponent** before deciding its move. Examples include games such as **chess, tic-tac-toe, and Rock–Paper–Scissors.**

In adversarial search, the AI explores possible future states of the game and selects the move that leads to the **best possible outcome**, assuming the opponent will also play optimally.

2. Game Tree Explanation

A **game tree** is a graphical representation of all possible moves in a game from the current state. It helps AI analyze different strategies before making a decision.

Components of a game tree:

- **Root Node:** Represents the current state of the game.
- **Branches:** Represent possible moves by the players.
- **Child Nodes:** Represent new game states after each move.
- **Leaf Nodes:** Represent final outcomes such as **win, loss, or draw.**

For the **Rock–Paper–Scissors** game:

- Each player has **three choices:** Rock, Paper, or Scissors.
- If the AI chooses one option, the opponent can choose any of the three options.
- Each combination leads to a different result.

Example outcomes:

- Rock vs Scissors → Win
- Rock vs Paper → Loss
- Rock vs Rock → Draw

All these possibilities are represented in the **game tree**, which the AI evaluates before making its decision.

3. Working of the Minimax Algorithm

The **Minimax algorithm** is a decision-making algorithm used in adversarial search to determine the **best move** for the AI player.

Basic idea:

- The **Max player (AI)** tries to **maximize the score.**
- The **Min player (opponent)** tries to **minimize the score.**

Steps of the Minimax Algorithm:

1. Construct the **game tree** for all possible moves.
2. Assign **utility values** to the leaf nodes.
   - Win = +1
   - Draw = 0
   - Loss = −1
3. Evaluate the tree from the bottom (leaf nodes) upward.
4. At **Max nodes**, choose the **maximum value.**
5. At **Min nodes**, choose the **minimum value.**
6. The value at the **root node** determines the best move for the AI.

Thus, the algorithm ensures that the AI chooses the move that gives the **best worst-case outcome.**

4. Alpha–Beta Pruning Optimization

Alpha–Beta pruning is an optimization technique used to **reduce the number of nodes evaluated in the minimax algorithm.**

It works by **eliminating branches of the game tree that cannot affect the final decision.**

Two important parameters are used:

- **Alpha (α):** The best value that the **maximizing player** can guarantee so far.
- **Beta (β):** The best value that the **minimizing player** can guarantee so far.

Pruning condition:

- If **α ≥ β**, the remaining branches of that node are **not explored** because they cannot improve the final result.

Advantages:

- Reduces unnecessary computations
- Improves efficiency
- Allows deeper game tree searches

5. Simple Example (Rock–Paper–Scissors)

Suppose the AI must choose between **Rock, Paper, and Scissors.**

Possible outcomes:

AI Move	Opponent Move	Result	Utility
Rock	Scissors	Win	+1
Rock	Paper	Loss	−1
Rock	Rock	Draw	0
Paper	Rock	Win	+1
Paper	Scissors	Loss	−1
Paper	Paper	Draw	0
Scissors	Paper	Win	+1
Scissors	Rock	Loss	−1
Scissors	Scissors	Draw	0

Using the **minimax algorithm**, the AI evaluates these outcomes assuming the opponent chooses the move that **minimizes AI's score.**

During evaluation, **alpha–beta pruning** may stop exploring certain branches once it becomes clear that they **cannot produce a better result** than already discovered moves.

6. Complexity Analysis

Minimax Algorithm

- **Time Complexity:** $O(b^d)$
  where **b = branching factor** and **d = depth of the tree**
- **Space Complexity:** $O(bd)$

Alpha–Beta Pruning

- **Best Case Time Complexity:** $O(b^{d/2})$
- **Worst Case Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(bd)$

Alpha–beta pruning significantly **reduces the number of nodes explored**, making the algorithm faster.

"Explain adversarial search in Artificial Intelligence using the game Rock–Paper–Scissors. Write the explanation in simple language with clear headings. Include the following sections: Definition, Game Tree Explanation, Working of Minimax Algorithm, Alpha-Beta Pruning Optimization, Example using the game, and Time and Space Complexity Analysis."

1. Definition of Adversarial Search

Adversarial search is a concept in **Artificial Intelligence (AI)** used in situations where **two players compete against each other**. In such problems, one player tries to **win or maximize their score**, while the other player tries to **reduce the opponent's advantage**.

This type of search is commonly used in **game-playing AI**, where the system must think about **its own moves as well as the opponent's possible moves**. Examples of such games include **chess, tic-tac-toe, and Rock–Paper–Scissors**.

In adversarial search, the AI explores different possible game situations and chooses the move that gives the **best possible result**, assuming the opponent will also play in the best possible way.

2. Game Tree Explanation

A **game tree** is a diagram that shows **all possible moves in a game** from the current position.

Important parts of a game tree:

- **Root Node:** The current state of the game.
- **Branches:** Possible moves that players can make.
- **Child Nodes:** New game states after each move.
- **Leaf Nodes:** Final outcomes such as **win, loss, or draw**.

In **Rock–Paper–Scissors**, each player has **three choices**:

- Rock
- Paper
- Scissors

If the AI chooses **Rock**, the opponent can choose:

- Rock → Draw
- Paper → AI loses
- Scissors → AI wins

All these possible outcomes form the **game tree**, which helps the AI evaluate which move is best.

3. Working of the Minimax Algorithm

The **Minimax algorithm** is a method used in adversarial search to find the **best move for the AI player**.

The main idea is:

- The **Max player (AI)** tries to **maximize the score**.
- The **Min player (opponent)** tries to **minimize the score**.

Steps of the Minimax Algorithm:

1. Generate all possible moves and create the **game tree**.
2. Assign **utility values** to the final outcomes:
   - Win = +1
   - Draw = 0
   - Loss = −1
3. Start evaluating the tree from the **bottom (leaf nodes)**.
4. At **Max nodes**, select the **highest value**.
5. At **Min nodes**, select the **lowest value**.
6. Continue this process until the **root node** is reached.

The move with the **best value at the root** is selected as the best move for the AI.

4. Alpha–Beta Pruning Optimization

Alpha–Beta pruning is an improvement to the minimax algorithm that **reduces the number of nodes evaluated in the game tree.**

It works by **skipping branches that cannot affect the final decision.**

Two important values are used:

- **Alpha (α):** The best value that the **maximizing player** can achieve so far.
- **Beta (β):** The best value that the **minimizing player** can achieve so far.

If at any point **α ≥ β**, the algorithm stops exploring that branch because it will not improve the final result. This process is called **pruning.**

Benefits of Alpha–Beta Pruning:

- Reduces unnecessary calculations
- Makes the algorithm faster
- Allows deeper search in complex games

5. Example Using Rock–Paper–Scissors

Suppose the AI needs to choose between **Rock, Paper, or Scissors.**

Possible outcomes are shown below:

AI Move	Opponent Move	Result	Utility Value
Rock	Scissors	Win	+1
Rock	Paper	Loss	−1
Rock	Rock	Draw	0
Paper	Rock	Win	+1
Paper	Scissors	Loss	−1
Paper	Paper	Draw	0
Scissors	Paper	Win	+1
Scissors	Rock	Loss	−1
Scissors	Scissors	Draw	0

Using **Minimax**, the AI evaluates all these possibilities while assuming the opponent will choose the move that gives the **worst outcome for the AI.**

With **Alpha–Beta pruning**, some branches of the tree may be ignored if they **cannot give a better result** than the already evaluated moves.

6. Time and Space Complexity Analysis

Minimax Algorithm

- **Time Complexity:** $O(b^d)$
  where
  b = branching factor (number of possible moves)
  d = depth of the game tree
- **Space Complexity:** $O(bd)$

Alpha–Beta Pruning

- **Best Case Time Complexity:** $O(b^{d/2})$
- **Worst Case Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(bd)$

Alpha–Beta pruning can **significantly reduce the number of nodes evaluated**, making the search process more efficient.

CONCLUSION:

_____

_____

_____

_____

DISCUSSION QUESTIONS:

1. What is prompt engineering?

   _____

   _____

   _____

   _____

2. Why is prompt refinement important?

   _____

   _____

   _____

   _____

3. What is Alpha-Beta pruning?

   _____

   _____

   _____

   _____

4. Difference between weak and strong prompts?

   _____

   _____

   _____

   _____

REFERENCES:

- https://learnpromptengineering.dev
- https://www.linkedin.com/pulse/40-prompt-engineering-resources-upskilling-spencer-taylor-xje0e/
- https://justprompting.in/